

E-Migration Assist — Functional Spec

E-Migration Assist — Functional Specification

Product owner: eRide Technologies **Document version:** 1.0 — May 5, 2026 **Status:** Pre-launch pilot — fully functioning

1. Product summary

E-Migration Assist is a pre-launch lead-capture and case-handover system for South African immigration assistance. It lets prospective clients self-assess their situation through a structured questionnaire, issues them a private reference number, and routes the lead into a lightweight operator CRM where it can be triaged, contacted, converted into a case, and progressed through its lifecycle until closure.

The product is split into three audiences:

- 1. **Public visitors** — anyone can access the landing page, complete the assessment, and check the status of their reference number. No account, login, or payment required.
- 2. **Returning leads** — people with a reference number can look up their status and (in future) supplementary documents.
- 3. **Operators (admin)** — the eRide team works the lead funnel and the case lifecycle through a token-protected admin console.

The system is **fully functioning end-to-end** today: every flow described below has been smoke-tested against the live API and verified in the browser.

2. User personas

Persona	Goal	Primary surface
Prospective applicant	Find out where they stand on a SA immigration matter	/, /assessment, /status
Returning applicant	Check progress against a reference number they were issued	/status
Operator (eRide staff)	Triage incoming leads, contact them, convert qualified leads into cases, progress cases	/admin, /admin/lead/:id, /admin/case/:caseId

3. Information architecture

Public:
/
/assessment 5-step assessment form
/thank-you/:ref Confirmation screen with reference number
/status Reference-number lookup form
└ result view (publicLabel only — no PII)

Admin (requires token):
/admin Lead dashboard (table + filters + inline edit)
/admin/lead/:id Full lead detail (notes, status, engagements)
/admin/case/:caseId Case detail + lifecycle dropdown

Every page renders the shared BrandHeader (eRide logo + “E-Migration Assist” wordmark, linking back to /).

4. Core user flows

4.1 Public assessment & lead creation

1. Visitor lands on / → sees the hero, the three explainer cards (“What this is”, “Who it’s for”, “What happens next”), and the pre-launch activity counters.
2. Clicks **Start Preliminary Assessment** → routed to /assessment.
3. Steps through a **5-step form**:
 - **Step 1 — Basic info**: nationality, current country, “I am currently inside South Africa” toggle.
 - **Step 2 — Situation**: picks one of the canonical situations (visa expired, overstay, lost documents, undocumented arrival, other).
 - **Step 3 — Urgency**: date / urgency band.
 - **Step 4 — Contact**: name, email, optional WhatsApp number with country auto-detect.
 - **Step 5 — Consent**: explicit checkbox; submit button enabled only after acceptance.
4. On submit, the form POSTs to /api/leads. The server:
 - Validates with Zod (returns 400 with field errors on failure).
 - Computes lead_score, lead_category, internal_classification.
 - Generates a unique EMA-XXXXXXXX-XXXX reference number.
 - Inserts the row with lead_status = “new”.
 - Optionally enqueues an outbound confirmation message (email or WhatsApp depending on preferred_contact_channel) — non-blocking.
5. Browser is redirected to /thank-you/:reference, which displays the reference number and the option to upload supporting documents.

4.2 Optional document upload (post-submission)

1. From /thank-you/:ref, the visitor can drag/drop or pick files.
2. Each file goes through POST /api/leads/:id/documents:
 - Server validates **magic bytes** (file-type) — only PDF, JPG, PNG accepted.
 - Server enforces **10 MB max** size.
 - Files are streamed to Replit Object Storage with a UUID-prefixed storage key (no enumeration vector).
 - A lead_documents row is inserted.
3. If validation fails, the upload UI shows a per-file error toast.
4. Documents are never accessible to the public; only admins can fetch them, and only via a server-side proxy that re-checks the admin token.

4.3 Public status lookup

1. Visitor goes to /status, types their reference number, hits Check.
2. Frontend calls GET /api/public/status/:reference.
3. Server returns **only**:

```

{
  "referenceNumber": "EMA-...",
  "publicLabel": "Under preliminary review",
  "createdAt": "...",
  "documentsUploaded": 2
}

```

4. The publicLabel is a generalised, non-PII description derived from the lead's actual status (e.g. all of new, reviewing, contacted map to the same outward label so operators' workflow isn't visible).
5. Rate limiting is aggressive on this endpoint to defeat enumeration.

4.4 Operator: sign in to the admin console

1. Operator visits /admin.
2. If no admin token is in localStorage, a sign-in card appears with a single password input. The token is stored in localStorage on submit and the dashboard loads.
3. Any admin API call that returns 401 clears the stored token and bounces the operator back to the sign-in card. The operator can sign out manually via the **Sign out** action.

4.5 Operator: triage the lead funnel

The dashboard at /admin shows a sortable, filterable table of leads with columns:

Column	Notes
Reference	Links to /admin/lead/:id
Created	Relative date
Name & contact	Full name + email + WhatsApp
Category	visa_expired / overstay / etc.
Status	Inline <Select> dropdown — backward options disabled
Priority	Inline dropdown
Action	Contextual: Contact, Convert to Case, or Open Case

Filters: status, priority, search by name/email/reference.

Inline status edit: - Operator opens the dropdown → only same-or-forward statuses are enabled (others are visibly disabled with a "Forward-only funnel — cannot regress" tooltip). - Selecting a value PATCHes /api/admin/leads/:id optimistically; on failure (409 or network) the prior value is restored.

Contact quick-action: - The Contact button opens WhatsApp / email with a pre-filled message. - For leads strictly upstream of contacted (i.e. new or reviewing), clicking Contact also auto-PATCHes the lead to contacted. Lateral or downstream leads are left alone.

4.6 Operator: convert a lead into a case

1. When a lead reaches ready_for_case, the row's action button becomes **Convert to Case**.
2. Clicking Convert PATCHes the lead to converted. The server:
 - Validates the forward-only funnel.
 - Calls ensureCaseForLead() which INSERT ... ON CONFLICT DO NOTHING RETURNINGS into lead_cases.
 - Returns the updated lead **including** caseId.
3. The frontend deep-links to /admin/case/:caseId from the response.
4. **Idempotent:** clicking Convert twice (or PATCHing a converted lead with a notes-only update) always surfaces the same caseId. No duplicate cases will ever be created — the DB unique index on lead_cases.lead_id is the durable guarantee.
5. Once lead_status === converted, the action button changes to **Open Case** (link only, no PATCH).

4.7 Operator: progress a case through its lifecycle

The case detail page at `/admin/case/:caseId` shows:

- The case reference number.
- The current case status (badge + dropdown).
- The current lead status (badge).
- The next-step hint derived from status.
- The original lead snapshot (read-only).
- The internal notes from the lead record.

Status dropdown: - Lists all canonical case statuses in order: Initiated → In Review → Documents Requested → Submitted → Closed. - The current value is selected; backwards options are disabled with a “Forward-only lifecycle — cannot regress” tooltip. - Selecting a forward value PATCHes `/api/admin/cases/:caseId` optimistically. On 409 or error, the prior value is restored and a toast surfaces the server’s explanation.

4.8 Operator: send a manual update email

From the lead detail page, the operator can compose a manual update email. The body is screened for forbidden phrases (legal-jargon guard) before sending via Resend. The send is recorded in `lead_engagements` with the provider message id and final delivery status.

4.9 Inbound WhatsApp from a lead

1. Lead replies to a WhatsApp message; Twilio POSTs to `/api/webhooks/whatsapp`.
 2. The endpoint validates the Twilio HMAC signature (when `WHATSAPP_APP_SECRET` is configured), then immediately returns 200.
 3. The message is stored in `case_messages` with the lead id resolved from the canonical WhatsApp number.
 4. If the body matches the deterministic keyword set (done, uploaded, sent), the message gets `intent = "task_complete_signal"` so operators can quickly find “I’m done” replies.
 5. Operators see inbound messages on the lead detail page.
-

5. Reference data — enums

5.1 Lead status (`prelaunch_leads.lead_status`)

new → reviewing → contacted → qualified → ready_for_case → converted → closed

- Forward-only at the API; same-status PATCH is a no-op.
- Backward PATCH → HTTP 409.

5.2 Lead priority

high | medium | low

5.3 Case status (`lead_cases.status`)

initiated → in_review → documents_requested → submitted → closed

- Forward-only at the API; same-status PATCH is a no-op.
- Backward PATCH → HTTP 409.

5.4 Document MIME types accepted

`application/pdf`, `image/jpeg`, `image/png` — checked by magic bytes, not just header. Max 10 MB per file.

5.5 Communication channels

email, whatsapp — set per lead in `preferred_contact_channel` and respected by the dispatcher.

6. Non-functional behaviour

Concern	Behaviour
Auth	Single admin token (ADMIN_EMAIL_TOKEN); constant-time compared. Server fails closed (503) if unset.
PII	Stripped from public endpoints. Public lookup never returns email/whatsapp/notes/status/score/caseld.
Rate limiting	Public endpoints (especially status lookup) are tightly throttled to defeat enumeration.
Idempotency	Lead → case conversion is DB-enforced unique on lead_id. Same-status PATCHes (lead and case) are 200 no-ops.
Concurrency	Both forward-only guards are encoded atomically in the SQL UPDATE ... WHERE predicate — no read-then-write race window.
Failure modes	Outbound email/WhatsApp delivery is non-blocking on lead submission; failures are recorded in lead_engagements but never break user flows.
Cooldowns	Resubmissions trigger a fresh confirmation only after a 1-minute per-channel cooldown.
Branding	Dark-navy + teal eRide palette throughout, BrandHeader on every page.
Logging	Server uses req.log (pino) with PII redaction. console.log is forbidden in server code.

7. Acceptance — what “fully functioning” means today

The pre-launch pilot has passed the following end-to-end go-live checklist (verified against the live deployment):

#	Check	Status
1	Public / and /assessment load over HTTPS	☐
2	A submission creates a lead and returns a reference number	☐
3	/thank-you/:ref displays the reference and accepts uploads	☐
4	/status returns only public-safe fields (no PII)	☐
5	/admin requires the token; all admin endpoints reject 401 unauth	☐
6	Lead dashboard loads with token, filters, and inline edits work	☐
7	Status can be moved forward through the funnel; backwards blocked with 409	☐
8	Contact button opens WA/email and auto-advances new/reviewing to contacted	☐
9	Convert creates exactly one case (DB unique constraint enforced under concurrent PATCHes)	☐
10	Open Case deep-links to /admin/case/:caseld	☐
11	Case status dropdown advances forward; backwards blocked with 409	☐
12	Inbound WhatsApp messages are stored and surfaced on the lead detail page	☐
13	Outbound email + WhatsApp confirmations are sent and audited in lead_engagements	☐
14	pnpm run typecheck passes across all four artifacts and shared libs	☐
15	Branding (logo + navy/teal palette) is applied to every page	☐

8. Out of scope for this release

- Payments / Paystack integration.
- Odoo / HubSpot / Jira sync.
- Multi-tenant (per-firm) scoping.
- Background job runner / outbox pattern.
- Mobile app (the existing screens are responsive but not native).
- Public registration with passwords (no user accounts; reference number is the only durable handle for a lead).

These are all on the platform roadmap visible in the architecture

diagram but are not implemented in the current codebase.